

Relatório Técnico

Controlador de Vending Machine em SystemVerilog

Trilha: RTL Design – Módulo: Projeto de Controlador Digital

Aluno: Apolo de Lima Silva

Equipe: Individual

Data: 10 de julho de 2026

1. Link para o repositório GitHub do projeto

O projeto foi organizado conforme a estrutura na Figura 1 abaixo:

```
grupo_NN_vending/  
├─ Makefile  
├─ rtl/  
│   ├── vending_pkg.sv  
│   ├── credit_reg.sv  
│   ├── memory.sv  
│   ├── comparator.sv  
│   ├── subtractor.sv  
│   ├── control_unit.sv  
│   └─ vending_top.sv  
├─ sim/  
│   └─ tb_vending.sv  
└─ synth/  
    ├── synth.tcl  
    ├── vending.sdc  
    └─ reports/
```

Figura 1: Estrutura do projeto.

O repositório correspondente é:

https://github.com/apoloSilva/grupo_NN_vending

2. Diagrama de blocos

A arquitetura foi implementada por meio de uma unidade de controle (`control_unit.sv`) e módulos que compõem o caminho de dados (`credit_reg.sv`, `memory.sv`, `comparator.sv` e `subtractor.sv`). A Figura 2 apresenta o diagrama em blocos do sistema desenvolvido, com identificação em vermelho do caminho de *clock* esperado.

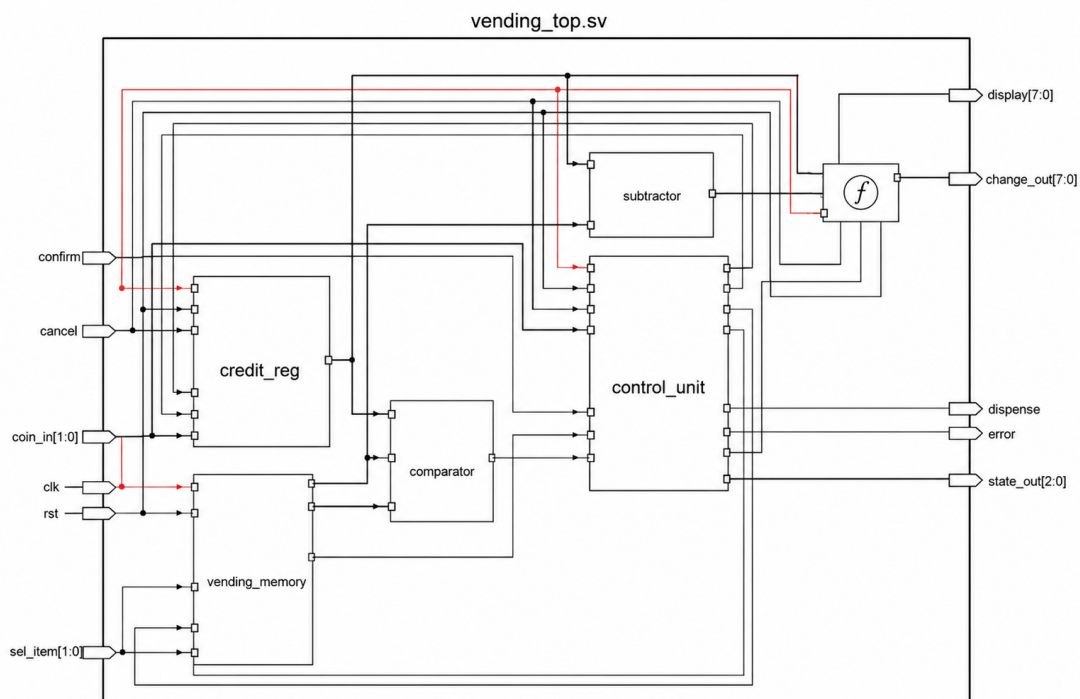


Figura 2: Diagrama geral em blocos do controlador de vending machine.

3. Diagrama de estados

A FSM possui seis estados codificados em três bits. A Figura 3 apresenta o diagrama de estados da máquina de estados implementada na unidade de controle, no arquivo `control_unit.sv`.

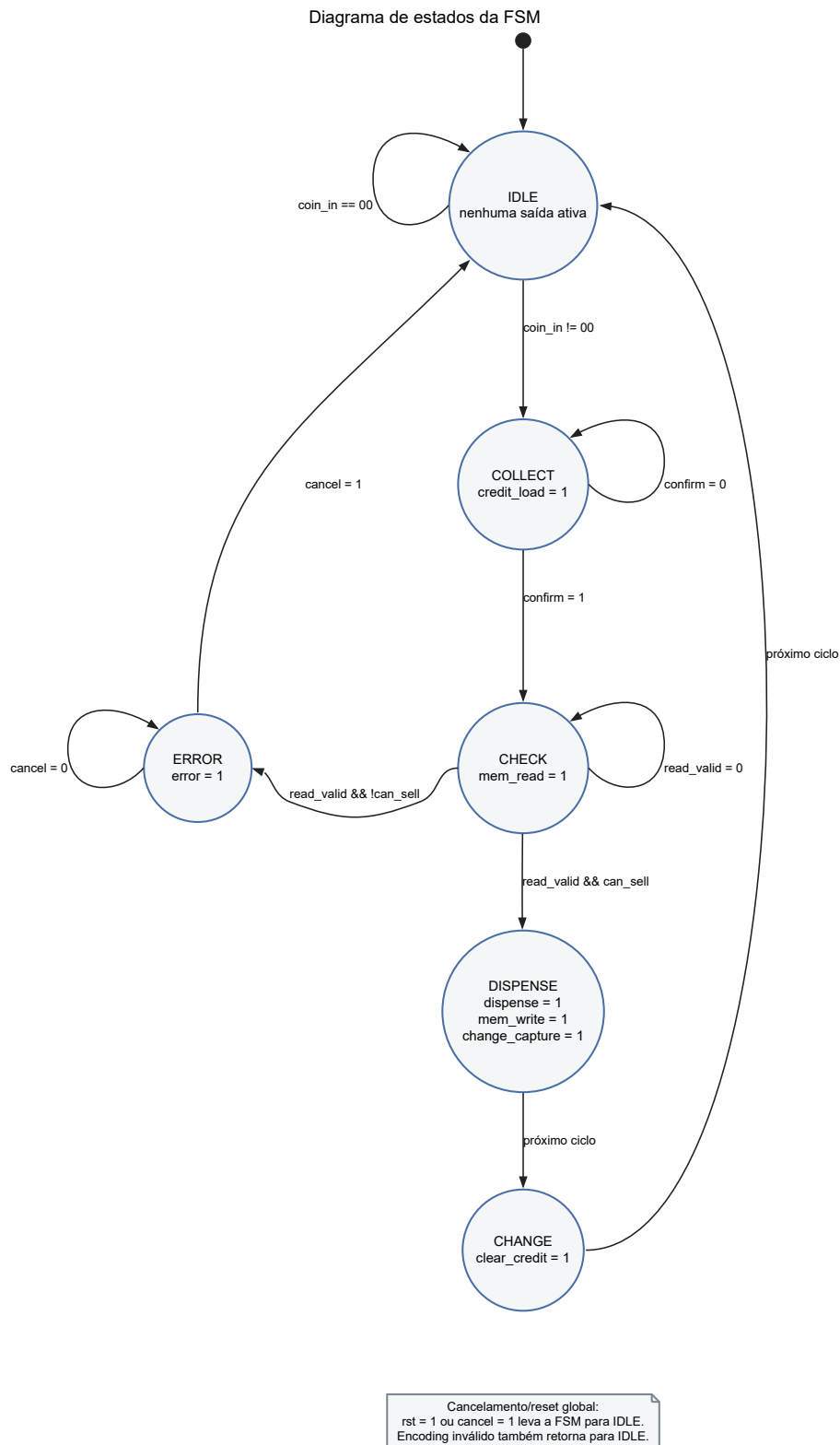


Figura 3: Diagrama de transição de estados e saídas de Moore.

4. Descrição dos módulos

Módulo	Decisão de projeto e justificativa
vending_pkg.sv	Centraliza os códigos dos seis estados e a função <code>coin_to_value</code> . Dessa forma, os valores de R\$0,25, R\$0,50 e R\$1,00 são utilizados de modo consistente em todo o projeto.
credit_reg.sv	Implementa um registrador síncrono de 8 bits. A prioridade é <code>rst > cancel > limpeza em CHANGE > acumulação</code> , garantindo devolução e limpeza determinísticas.
memory.sv	Modela quatro palavras de 16 bits, com <code>price</code> nos oito bits mais significativos e <code>stock</code> nos menos significativos. A interface de leitura é registrada e <code>read_valid</code> explicita a latência de um ciclo.
comparator.sv / subtractor.sv	Mantém o caminho aritmético puramente combinacional: <code>can_sell=(credit>=price)&&(stock>0)</code> e <code>change=credit-price</code> . A FSM é responsável por decidir quando utilizar cada resultado.
control_unit.sv	Usa um bloco <code>always_ff</code> para o estado e um bloco <code>always_comb</code> para o próximo estado e as saídas.
vending_top.sv	Instancia e interconecta todos os módulos. O primeiro crédito é aceito na borda que provoca <code>IDLE→COLLECT</code> , evitando perder a primeira moeda. <code>change_out</code> é registrado na transição <code>DISPENSE→CHANGE</code> ou durante <code>cancel</code> .

Tabela 1: Organização do RTL e decisões principais.

5. Waveforms anotadas dos cenários de teste

As capturas de forma de onda foram extraídas do Verdi a partir do arquivo `waves.fsdb`. Foram utilizadas as janelas temporais impressas pelo próprio testbench: reset inicial de 0 a 25 ns, cenário 1 de 25 a 105 ns, cenário 2 de 105 a 180 ns, cenário 3 de 180 a 240 ns e cenário 4 de 240 a 625 ns. Essas janelas também aparecem na validação FAIL/PASS de cada cenário, realizada com a função `check` no terminal, como será visto na próxima seção.

Em todas as capturas, o sinal `clk` foi utilizado como referência temporal. As entradas observadas foram `rst`, `coin_in`, `sel_item`, `confirm` e `cancel`. As saídas analisadas foram `state_out`, `display`, `dispense`, `change_out` e `error`.

No reset inicial, observa-se o sinal `rst` ativo durante dois ciclos de `clock`, inicializando a FSM em `IDLE` e zerando o crédito, o troco e os sinais de controle.

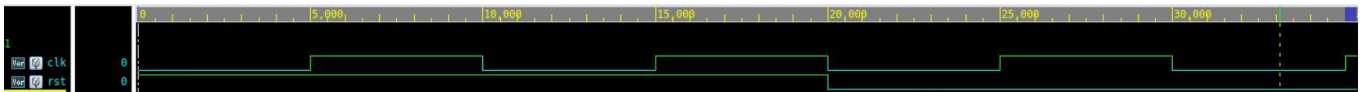


Figura 4: Waveform do reset inicial por dois ciclos de `clock`, mostrando a inicialização da FSM em `IDLE` e o crédito igual a zero.

No cenário 1, observa-se a inserção de R\$1,00 para a compra do café, a passagem pelos estados `DISPENSE` e `CHANGE`, a ativação de `dispense` por um ciclo e a geração de 75 centavos de troco.

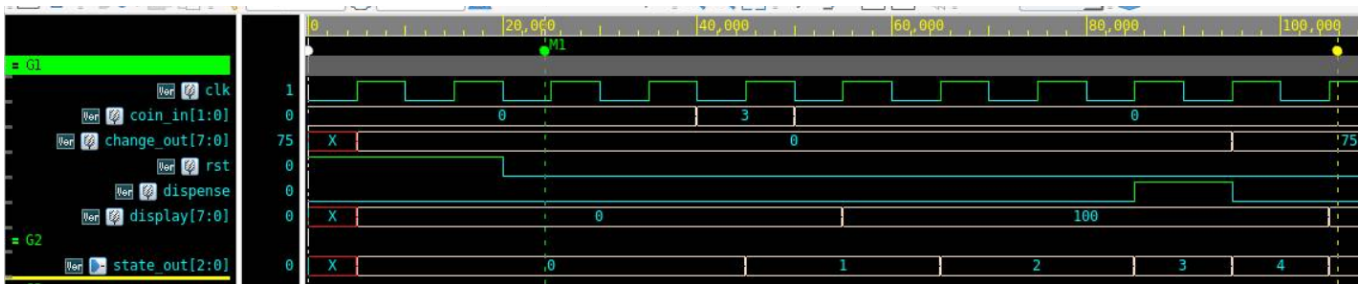


Figura 5: Cenário 1: compra bem-sucedida de café com R\$1,00 e geração de troco de 75 centavos.

No cenário 2, observa-se a tentativa de compra do snack com apenas 25 centavos, fazendo a FSM entrar em `ERROR`. Após o acionamento de `cancel`, o valor inserido é devolvido e o sistema retorna para `IDLE`.

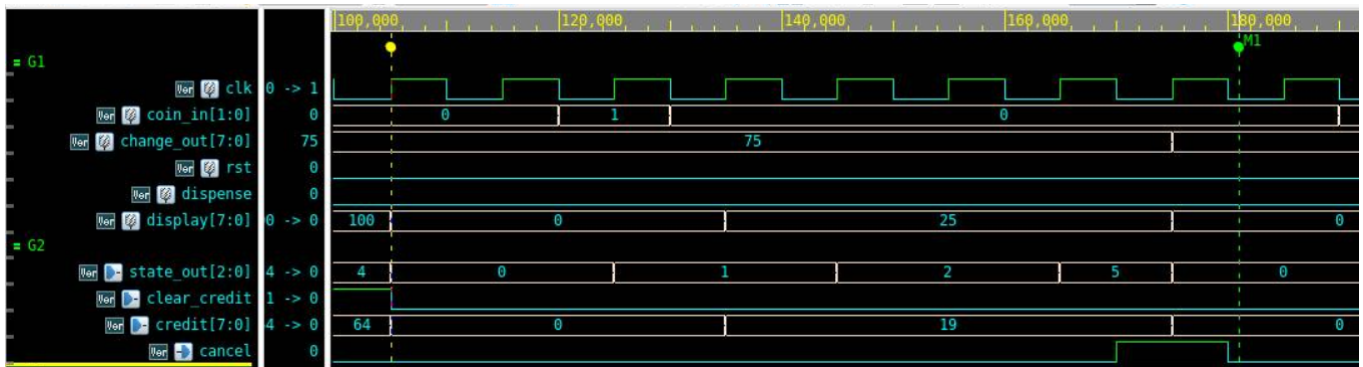


Figura 6: Cenário 2: tentativa de compra do snack com crédito insuficiente e posterior cancelamento.

No cenário 3, observa-se o acúmulo de 200 centavos após a inserção de duas moedas de R\$1,00. Com o acionamento de `cancel`, o valor integral é transferido para `change_out` e o crédito é zerado.

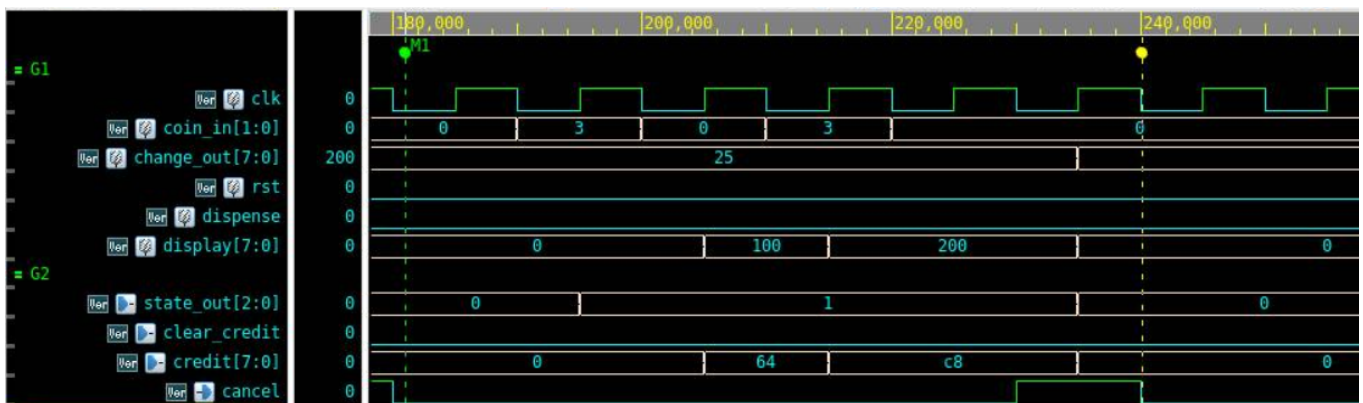


Figura 7: Cenário 3: cancelamento da operação com devolução integral dos R\$2,00 acumulados.

No cenário 4, observam-se quatro novas vendas bem-sucedidas de café, com decremento progressivo do estoque e geração de 75 centavos de troco em cada operação. Na tentativa seguinte, o estoque zerado impede a venda e leva a FSM ao estado `ERROR`.

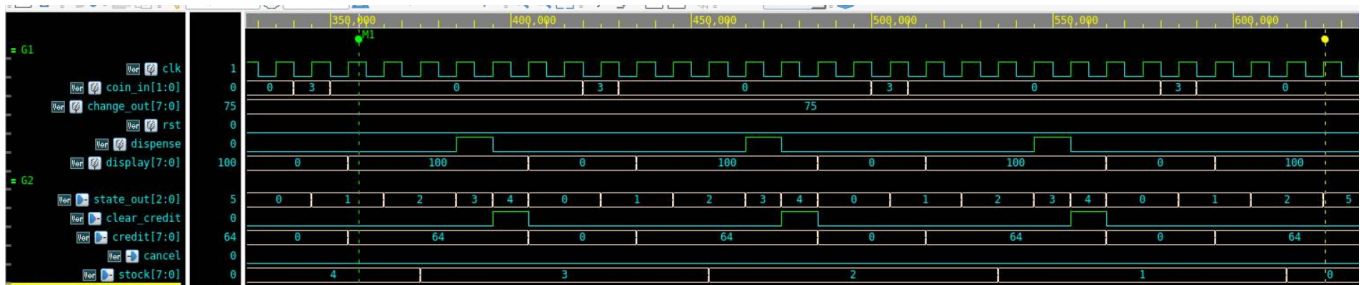


Figura 8: Cenário 4: quatro vendas adicionais de café, esgotamento do estoque e erro na sexta tentativa de compra.

6. Saída do testbench

O testbench executa as verificações automaticamente por meio da tarefa `check(expected, actual, label)`. Cada comparação imprime `PASS` ou `FAIL`, e o resultado final contabiliza o número total de verificações e falhas. A execução abaixo cobre o reset inicial e os quatro cenários obrigatórios.

```
Reset inicial por 2 ciclos de clock
[PASS] Reset: FSM inicia em IDLE | expected=0 actual=0
[PASS] Reset: credito inicial igual a zero | expected=0 actual=0
[TIME] Apos reset inicial: 25 ns

=====
CENARIO 1: compra de cafe com troco
=====
[PASS] S1: FSM entra em DISPENSE | expected=3 actual=3
[PASS] S1: dispense ativo em DISPENSE | expected=1 actual=1
[PASS] S1: FSM entra em CHANGE | expected=4 actual=4
[PASS] S1: dispense dura apenas um ciclo | expected=0 actual=0
```

```
[PASS] S1: troco igual a 75 centavos | expected=75 actual=75
[PASS] S1: FSM retorna para IDLE | expected=0 actual=0
[PASS] S1: credito zerado ao final da venda | expected=0 actual=0
[TIME] Apos cenario 1: 105 ns

=====
CENARIO 2: credito insuficiente
=====
[PASS] S2: FSM entra em ERROR | expected=5 actual=5
[PASS] S2: error permanece ativo | expected=1 actual=1
[PASS] S2: cancel retorna para IDLE | expected=0 actual=0
[PASS] S2: cancel devolve 25 centavos | expected=25 actual=25
[PASS] S2: credito zerado apos cancelamento | expected=0 actual=0
[TIME] Apos cenario 2: 180 ns

=====
CENARIO 3: cancelamento com R$2,00
=====
[PASS] S3: credito acumulado igual a 200 | expected=200 actual=200
[PASS] S3: FSM retorna para IDLE | expected=0 actual=0
[PASS] S3: cancel devolve 200 centavos | expected=200 actual=200
[PASS] S3: credito zerado apos cancelamento | expected=0 actual=0
[TIME] Apos cenario 3: 240 ns

=====
CENARIO 4: estoque de cafe zerado
=====
[PASS] S4: venda 2 entra em DISPENSE | expected=3 actual=3
[PASS] S4: venda 2 gera dispense | expected=1 actual=1
[PASS] S4: venda 2 entra em CHANGE | expected=4 actual=4
[PASS] S4: venda 2 devolve 75 centavos | expected=75 actual=75
[PASS] S4: venda 2 retorna para IDLE | expected=0 actual=0
[PASS] S4: venda 3 entra em DISPENSE | expected=3 actual=3
[PASS] S4: venda 3 gera dispense | expected=1 actual=1
[PASS] S4: venda 3 entra em CHANGE | expected=4 actual=4
[PASS] S4: venda 3 devolve 75 centavos | expected=75 actual=75
[PASS] S4: venda 3 retorna para IDLE | expected=0 actual=0
[PASS] S4: venda 4 entra em DISPENSE | expected=3 actual=3
[PASS] S4: venda 4 gera dispense | expected=1 actual=1
[PASS] S4: venda 4 entra em CHANGE | expected=4 actual=4
[PASS] S4: venda 4 devolve 75 centavos | expected=75 actual=75
[PASS] S4: venda 4 retorna para IDLE | expected=0 actual=0
[PASS] S4: venda 5 entra em DISPENSE | expected=3 actual=3
[PASS] S4: venda 5 gera dispense | expected=1 actual=1
[PASS] S4: venda 5 entra em CHANGE | expected=4 actual=4
[PASS] S4: venda 5 devolve 75 centavos | expected=75 actual=75
[PASS] S4: venda 5 retorna para IDLE | expected=0 actual=0
[PASS] S4: sexta tentativa entra em ERROR | expected=5 actual=5
[PASS] S4: error ativo pois o estoque esta zerado | expected=1 actual=1
[TIME] Apos cenario 4: 625 ns

=====
RESULTADO DO TESTBENCH: PASS (40 checks, 0 failures)
=====
```

A saída mostra 40 verificações e nenhuma falha. O cenário 4 aparece a partir da venda 2 porque a simulação foi executada de forma sequencial: o cenário 1 já consumiu uma unidade de café. Assim, as vendas 2 a 5 completam o esgotamento do estoque, e a tentativa seguinte corresponde à sexta compra total de café. O log completo da simulação está disponível no repositório.

7. Resultados de síntese e exploração de constraints

A síntese foi executada no Synopsys Design Compiler (DC), utilizando a biblioteca de células-padrão `saed32rvt_tt1p05v25c.db`, fornecendo as células utilizadas no mapeamento do circuito, incluindo a célula `NBUFFX2_RVT`, empregada para modelar o acionamento das entradas. A biblioteca sintética `dw_foundation.sldb` também foi incluída, disponibilizando os componentes do DesignWare utilizados pelo DC durante a síntese de operadores aritméticos e lógicos.

O comando `check_design` foi executado antes da síntese e não indicou erros fatais. Ao final do processo, foram geradas a netlist sintetizada `vending_top_syn.v`, a representação `vending_top_syn.ddc` e os relatórios de área, timing, potência e violações de constraints.

As constraints foram definidas no arquivo `synth/vending.sdc`, incluindo clock, incerteza de clock, atrasos de entrada e saída, célula de acionamento das entradas e carga capacitiva nas saídas.

A análise inicial foi realizada com um período de clock de 20 ns, correspondente a uma frequência de 50 MHz. Nessa condição, o pior caminho de setup apresentou slack positivo de 15,43 ns, indicando que a constraint inicial foi atendida.

O pior caminho parte da entrada `coin_in[0]` e termina no registrador `u_credit_reg/credit_reg[7]`. O caminho atravessa a lógica de controle da inserção da moeda e a cadeia de somadores utilizada para atualizar o crédito acumulado.

Item	Resultado inicial
Período do clock	20 ns
Frequência do clock	50 MHz
Slack de setup	+15,43 ns
Resultado de setup	Atendido (<i>MET</i>)
Caminho inicial	coin_in[0] até u_credit_reg/credit_reg[7]
Número de células	280
Células combinacionais	206
Células sequenciais	62
Área combinacional	502,188546 u.a.
Área não combinacional	410,696717 u.a.
Área total de células	912,885263 u.a.
Área total	1080,284448 u.a.

Tabela 2: Resultados da síntese inicial com um período de 20 ns.

7.1 Exploração do período do clock

Para determinar a frequência máxima de operação, o período do clock foi reduzido progressivamente em passos de 2 ns. Para cada período, o projeto foi novamente sintetizado, registrando-se o pior slack de setup apresentado pelo comando `report_timing` e a área total apresentada pelo comando `report_area`.

A frequência correspondente a cada período foi calculada por:

$$f = \frac{1000}{T}, \quad (1)$$

em que T é o período em nanossegundos e f é a frequência em megahertz.

Período (ns)	Frequência (MHz)	Slack de setup (ns)	Área total (u.a.)	Resultado
20	50,00	+15,43	1080,284448	Atendido
18	55,56	+13,43	1080,284448	Atendido
16	62,50	+11,43	1080,284448	Atendido
14	71,43	+9,43	1080,284448	Atendido
12	83,33	+7,43	1080,284448	Atendido
10	100,00	+5,43	1080,284448	Atendido
8	125,00	+3,42	1083,566170	Atendido
6	166,67	+1,43	1085,960390	Atendido
4	250,00	-0,14	1427,170008	Violado

Tabela 3: Exploração do período do clock, do slack de setup e da área total.

O menor período para o qual o slack permaneceu não negativo foi de 6 ns, correspondente a uma frequência de aproximadamente 166,67 MHz. Nesse ponto, o slack de setup foi de +1,43 ns e a área total sintetizada foi de 1085,960390 u.a.

Ao reduzir o período para 4 ns, isto é, 2 ns abaixo do menor período atendido, o DC apresentou slack de -0,14 ns. Dessa forma, o timing passou a ser violado. A área total nessa condição foi de 1427,170008 u.a., indicando que o sintetizador aumentou a área na tentativa de atender à constraint mais restritiva.

Entre os períodos de 20 ns e 10 ns, a área total permaneceu constante. Nos períodos de 8 ns e 6 ns, foram observados pequenos aumentos de área. Entretanto, para o período de 4 ns, a área aumentou aproximadamente 31,42% em relação à síntese de 6 ns. O número total de células também aumentou de 283 para 431, mostrando que o DC realizou um mapeamento mais agressivo na tentativa de reduzir os atrasos.

Consequentemente, a frequência máxima obtida para o projeto, considerando a exploração em passos de 2 ns, foi:

$$f_{\max} = \frac{1000}{T_{\min}} = \frac{1000}{6} = 166,67 \text{ MHz}. \quad (2)$$

7.2 Comparação da opção `-no_autoungroup`

Após encontrar o menor período com slack não negativo, a síntese foi repetida retirando-se a opção `-no_autoungroup` do comando `compile_ultra`. Essa comparação permite verificar se a eliminação automática da hierarquia produz alguma alteração no mapeamento, na área e no desempenho temporal.

Configuração	Período (ns)	Slack (ns)	Área total (u.a.)	Células
Com <code>-no_autoungroup</code>	6	+1,43	1085,960390	283
Sem <code>-no_autoungroup</code>	6	+1,39	1033,349343	246

Tabela 4: Comparação da síntese com e sem `-no_autoungroup`.

A retirada da opção `-no_autoungroup` não alterou significativamente o desempenho temporal. O slack de setup foi reduzido de +1,43 ns para +1,39 ns, uma diferença de apenas 0,04 ns, mantendo o timing atendido para o período de 6 ns.

Por outro lado, a área total foi reduzida de 1085,960390 u.a. para 1033,349343 u.a., correspondendo a uma redução de aproximadamente 4,84%. O número total de células também diminuiu de 283 para 246. A quantidade de células combinacionais foi reduzida de 220 para 183, enquanto o número de células sequenciais permaneceu igual a 62.

Esse resultado ocorre porque, sem a preservação explícita da hierarquia, o DC pode realizar otimizações entre módulos, eliminar limites hierárquicos, compartilhar expressões equivalentes e remover lógica redundante. Neste projeto, essas otimizações reduziram a área sem comprometer o fechamento de timing para o menor período atendido.

8. Análise do caminho crítico

Para o menor período atendido na exploração principal, de 6 ns, o pior caminho de setup parte da entrada `coin_in[1]` e termina no registrador `u_credit_reg/credit_reg[7]`. O caminho apresenta tempo de chegada de 4,05 ns, tempo requerido de 5,48 ns e slack positivo de 1,43 ns. Esse caminho atravessa principalmente o módulo `credit_reg`. Após a entrada `coin_in[1]`, o sinal percorre a lógica de detecção e controle da inserção da moeda e a cadeia aritmética utilizada para somar o valor da moeda ao crédito acumulado.

Ao reduzir o período para 4 ns, o DC realizou um novo mapeamento e o pior caminho passou a partir da entrada `sel_item[0]` e terminar no registrador `u_memory/mem_reg[3]` [3]. Esse caminho atravessa o módulo `vending_memory`, incluindo a lógica de decodificação do endereço e de atualização da posição selecionada da memória. A mudança do caminho crítico mostra que o sintetizador conseguiu otimizar parcialmente o caminho de acumulação do crédito.

9. Conclusão

O projeto implementou um controlador de vending machine composto por uma FSM de Moore, memória síncrona, registrador acumulador de crédito e blocos combinacionais de comparação e subtração. Os quatro cenários obrigatórios foram executados por um testbench self-checking, resultando em 40 verificações aprovadas e nenhuma falha.

As principais dificuldades estiveram relacionadas ao alinhamento temporal dos sinais de entrada e dos sinais síncronos. Foi necessário garantir que a primeira moeda fosse amostrada corretamente pela FSM e pelo registrador de crédito, além de considerar a latência de um ciclo da memória antes da decisão de venda. A execução sequencial dos cenários também precisou ser considerada, pois a compra realizada no primeiro cenário alterou o estoque utilizado posteriormente no teste de estoque zerado.

Na síntese inicial, realizada com um período de 20 ns, o circuito apresentou slack de setup positivo de 15,43 ns e área total de 1080,284448 u.a. O período foi reduzido em passos de 2 ns, conforme solicitado, até que fosse observada uma violação. O menor período atendido foi de 6 ns, com slack positivo de 1,43 ns e área total de 1085,960390 u.a. Dessa forma, a maior frequência verificada pela exploração foi de aproximadamente 166,67 MHz.

Para o período de 4 ns, correspondente a 250 MHz, o slack tornou-se negativo, atingindo -0,14 ns. Nessa condição, o DC aumentou a área total para 1427,170008 u.a. e o número de células para 431, mas não conseguiu fechar o timing. O caminho crítico também mudou do acumulador de crédito para a lógica de seleção e atualização da memória.

A comparação realizada sem a opção `-no_autoungroup`, no período de 6 ns, apresentou slack de 1,39 ns e área total de 1033,349343 u.a. A remoção da opção reduziu a área em aproximadamente 4,84e provocando apenas uma pequena redução de 0,04 ns no slack.

De forma geral, a atividade permitiu colocar em prática os conceitos apresentados nos laboratórios e no curso remoto.